# Копирование, перемещение и переименование

## 1. Назначение работы

В этой работе выполняются три базовые операции с собственными тестовыми файлами: копирование, переименование и перемещение. Для каждой операции сначала формируется исходный и целевой путь, затем выполняется действие и обязательно проверяется фактический результат.
Работа рассчитана на 2 академических часа. Все действия выполняются в Python 3.x в среде Visual Studio Community только внутри рабочей папки проекта.

> Важно
> Все операции выполняются только с тестовыми файлами, созданными внутри собственной рабочей папки проекта. Не копируйте, не перемещайте и не переименовывайте системные файлы, файлы программ или документы других пользователей.

## 2. Используемое программное обеспечение

| Компонент | Назначение |
|---|---|
| Python 3.x | выполнение программы и работа со стандартной библиотекой |
| Visual Studio Community | создание, запуск и проверка Python-проекта |
| pathlib.Path | формирование путей, проверка exists()/is_file(), переименование через rename() |
| shutil | копирование copy2() и перемещение move() |

## 3. Порядок выполнения работы

### 3.1. Подготовка безопасной рабочей структуры

Сначала подготовьте отдельную тестовую структуру. Она нужна для того, чтобы все операции выполнялись только с файлами, созданными специально для этой работы.

1. Создайте или откройте Python-проект в Visual Studio Community.
2. Убедитесь, что файл main.py запускается без ошибок.
3. Выведите Path.cwd(), чтобы увидеть текущую рабочую папку.
4. Не используйте системные каталоги и чужие пользовательские файлы.

**Подготовка учебной структуры**

```
from pathlib import Path

base = Path.cwd() / "demo_ops"
source = base / "source"
copies = base / "copies"
archive = base / "archive"

source.mkdir(parents=True, exist_ok=True)
copies.mkdir(parents=True, exist_ok=True)
archive.mkdir(parents=True, exist_ok=True)

with open(source / "report.txt", "w", encoding="utf-8") as file:
    file.write("Учебный отчёт\n")
    file.write("Статус: подготовлен\n")

print("Рабочая структура подготовлена:", base)
```

После запуска внутри demo_ops появятся три папки. Файл report.txt создаётся только в source и используется далее как объект учебного примера.

**Ожидаемая структура до операций**

```
demo_ops
├── source
│   └── report.txt
├── copies
└── archive
```

Что показывает этот пример:
Path.cwd() задаёт безопасную рабочую область. Все три каталога создаются внутри demo_ops, а report.txt появляется только в source. Дальше можно проверять, как одна и та же файловая структура меняется после разных операций.

**Задание 1. Подготовить собственную структуру**

1. Создайте внутри Path.cwd() каталог file_lab.
2. Внутри file_lab создайте папки incoming, backup и ready.
3. Программно создайте incoming/notes.txt и incoming/data.txt. В каждом файле запишите минимум две строки текста.
4. Откройте file_lab в Проводнике и сделайте скриншот исходной структуры.

### 3.2. Копирование файла

Копирование создаёт второй файл с тем же содержимым, а исходный объект остаётся на прежнем месте. Для операции используется shutil.copy2().

**Минимальный синтаксис**

```
from shutil import copy2

copy2(source_path, destination_path)
```

**Пример: копирование report.txt**

```
from pathlib import Path
from shutil import copy2

base = Path.cwd() / "demo_ops"
source_file = base / "source" / "report.txt"
copy_file = base / "copies" / "report_copy.txt"

copy2(source_file, copy_file)

print("Исходный файл существует:", source_file.exists())
print("Копия существует:", copy_file.exists())
```

Разбор результата:
После copy2() исходный report.txt остаётся в source, а report_copy.txt появляется в copies. Значит, успешное копирование подтверждается тем, что после операции существуют оба пути.

**Задание 2. Скопировать notes.txt**

1. Сформируйте путь к incoming/notes.txt.
2. Сформируйте путь к будущей копии backup/notes_backup.txt.
3. Выполните копирование через copy2().
4. После операции выведите exists() для исходного файла и для копии.
5. Зафиксируйте результат скриншотом Проводника или консоли.

### 3.3. Переименование файла

Переименование изменяет имя файла, но не создаёт второй экземпляр. Если новый путь указывает на ту же папку, меняется только имя. Для операции используется Path.rename().

**Общая форма**

```
old_path.rename(new_path)
```

**Пример: report.txt → report_final.txt**

```
from pathlib import Path

base = Path.cwd() / "demo_ops"
old_path = base / "source" / "report.txt"
new_path = base / "source" / "report_final.txt"

old_path.rename(new_path)

print("Старое имя существует:", old_path.exists())
print("Новое имя существует:", new_path.exists())
```

Как проверить результат:
После rename() старый путь должен перестать существовать, а новый — начать существовать. Файл остаётся в source; меняется только его имя.

**Задание 3. Переименовать data.txt**

1. Сформируйте старый путь incoming/data.txt.
2. Сформируйте новый путь incoming/measurements.txt.
3. Выполните rename().
4. Проверьте оба пути через exists().
5. В отчёте кратко объясните, почему после операции старый и новый пути дают разные результаты.

### 3.4. Перемещение файла

Перемещение переносит файл из одного каталога в другой. После успешной операции исходный путь перестаёт содержать этот файл, а целевой путь начинает существовать. Для операции используется shutil.move().

**Минимальный синтаксис**

```
from shutil import move

move(source_path, destination_path)
```

**Пример: перемещение report_final.txt в archive**

```
from pathlib import Path
from shutil import move

base = Path.cwd() / "demo_ops"
source_file = base / "source" / "report_final.txt"
destination_file = base / "archive" / "report_final.txt"

move(source_file, destination_file)

print("В source остался файл:", source_file.exists())
print("В archive появился файл:", destination_file.exists())
```

Разбор результата:
В отличие от копирования, после move() остаётся один экземпляр файла — уже в новом каталоге. Проверка двух путей показывает именно изменение местоположения.

**Задание 4. Переместить measurements.txt**

1. Сформируйте исходный путь incoming/measurements.txt.
2. Сформируйте целевой путь ready/measurements.txt.
3. Выполните move().
4. Проверьте оба пути через exists().
5. Откройте incoming и ready в Проводнике и убедитесь, что файл находится только в ready.

### 3.5. Защита от случайной перезаписи

Перед копированием, перемещением или переименованием нужно проверить не только исходный, но и будущий путь. Если файл с таким именем уже существует, операция не выполняется автоматически.

**Проверка перед операцией**

```
if source_file.exists() and source_file.is_file():
    if not destination_file.exists():
        copy2(source_file, destination_file)
        print("Операция выполнена")
    else:
        print("Целевой файл уже существует")
else:
    print("Исходный файл не найден")
```

Почему эта проверка нужна:
Сначала подтверждается существование исходного файла, затем проверяется, свободно ли целевое имя. Это не даёт случайно выполнить операцию поверх уже существующего тестового файла.

**Задание 5. Добавить защитную проверку**

1. Для копирования notes.txt проверьте существование исходного файла.
2. Проверьте, отсутствует ли backup/notes_backup.txt.

3. Если копия уже существует, не выполняйте copy2() повторно и выведите понятное сообщение.
4. Запустите программу дважды и зафиксируйте различие результатов.

### 3.6. Проверка структуры после операций

После нескольких операций удобно вывести фактическое содержимое рабочих папок. Для этого можно использовать функцию, которая перебирает элементы через iterdir() и показывает тип каждого объекта.

#### Функция для вывода содержимого папки

```
def show_folder(folder):
    print("\n" + folder.name)
    for item in folder.iterdir():
        if item.is_file():
            print("[ФАЙЛ]", item.name)
        elif item.is_dir():
            print("[КАТАЛОГ]", item.name)
```

Что демонстрирует функция:
show_folder() ничего не изменяет. Она только перебирает непосредственное содержимое указанной папки и позволяет сверить фактическую структуру после операций.

#### Задание 6. Проверить file_lab

1. Выведите содержимое incoming, backup и ready.
2. Определите, какие файлы остались в incoming.
3. Укажите, какая копия находится в backup.
4. Укажите, какой файл был перемещён в ready.

## 4. Полный учебный пример

Ниже копирование, переименование и перемещение собраны в одной программе. Учебный пример использует структуру demo_ops. Самостоятельная работа ниже использует другие имена и другую структуру.

### Полный пример «Учебный архив»

```
from pathlib import Path
from shutil import copy2, move

base = Path.cwd() / "demo_ops"
source = base / "source"
copies = base / "copies"
archive = base / "archive"

source.mkdir(parents=True, exist_ok=True)
copies.mkdir(parents=True, exist_ok=True)
archive.mkdir(parents=True, exist_ok=True)

original = source / "report.txt"
with open(original, "w", encoding="utf-8") as file:
    file.write("Учебный отчёт\n")

copy_path = copies / "report_copy.txt"
if not copy_path.exists():
    copy2(original, copy_path)

renamed = source / "report_final.txt"
if original.exists() and not renamed.exists():
    original.rename(renamed)

archived = archive / "report_final.txt"
if renamed.exists() and not archived.exists():
    move(renamed, archived)

print("Копия:", copy_path.exists())
print("Файл в архиве:", archived.exists())
print("Старое имя:", original.exists())
```

Разбор учебного примера:
1) copy2() создаёт report_copy.txt, не удаляя исходный файл. 2) rename() меняет путь report.txt на report_final.txt в той же папке. 3) move() переносит report_final.txt в archive. 4) exists() после каждой операции подтверждает не сообщение программы, а фактическое состояние файлов.

## 5. Итоговая самостоятельная работа

Создайте программу «Организатор тестовых файлов». Она должна подготовить собственную структуру, выполнить три разные операции, проверить результат и вывести итоговое содержимое рабочих папок.

### Структура самостоятельной работы

```
workspace_ops
├── incoming
│   ├── task.txt
│   ├── config.txt
│   └── draft.txt
├── backup
└── ready
```

В incoming программно создайте три файла: task.txt, config.txt и draft.txt. В каждом файле должно быть минимум две строки.

### Что выполнить

1. Скопируйте task.txt в backup под именем task_backup.txt.
2. Переименуйте config.txt в settings.txt внутри incoming.
3. Переместите settings.txt из incoming в ready.
4. Перед каждой операцией проверьте существование исходного и целевого пути.
5. После каждой операции выведите понятное сообщение о результате.
6. Выведите содержимое incoming, backup и ready через iterdir().
7. Проверьте, что копирование оставило исходный файл, переименование изменило имя, а перемещение изменило каталог.
8. Сделайте скриншоты, подтверждающие итоговую структуру.

> Важно
> Имена и структура самостоятельной работы отличаются от учебного примера. Используйте разобранную последовательность действий, но не заменяйте workspace_ops, incoming, backup и ready именами demo_ops, source, copies или archive.

## 6. Самостоятельное изменение

Создайте в incoming дополнительный текстовый файл с собственным именем. Выполните с ним одну безопасную операцию: копирование, переименование или перемещение. Не повторяйте полностью уже выполненный сценарий — измените имя файла, целевой каталог или имя результата.

В отчёте зафиксируйте исходный путь, целевой путь, выполненную операцию и фактический результат.

## 7. Требования к отчёту

Отчёт оформляется в Microsoft Word. В нём должны быть видны не только фрагменты кода, но и фактические изменения файловой структуры.

| Раздел отчёта | Что приложить |
|---|---|
| Тема и цель | название работы и краткая цель |
| Исходная структура | скриншот file_lab или workspace_ops до операций |
| Копирование | фрагмент кода + скриншот результата |
| Переименование | фрагмент кода + скриншот результата |
| Перемещение | фрагмент кода + скриншот результата |
| Итоговая программа | полный код текстом с сохранением отступов |
| Самостоятельное изменение | код, исходный/целевой путь и подтверждение |
| Вывод | 3–5 предложений о выполненных операциях и результатах |

> Важно для отчёта
> Код итоговой программы вставляется в Word текстом с сохранением отступов. Каждый скриншот подпишите «Рисунок N — ...» и под ним кратко поясните, какая операция выполнена и что подтверждает изображение.

### 7.1. Таблица наблюдений

Заполните таблицу только по фактическим результатам своей программы.

| Наблюдение | Фактический путь / имя | Результат |
|---|---|---|
| Исходный файл для копирования | | |
| Путь копии | | |
| Старое имя переименования | | |
| Новое имя переименования | | |
| Исходный путь перемещения | | |
| Целевой путь перемещения | | |
| Самостоятельная операция | | |

## 8. Типичные ошибки

- исходный путь сформирован не к тому файлу;
- целевой файл уже существует, но проверка destination.exists() отсутствует;
- после move() проверяется копия, а не исходный путь;
- после rename() проверяется старое имя, но не новое;
- операция выполняется вне Path.cwd();
- copy2(), move() и rename() используются без проверки результата;
- скриншот показывает папку, но не подтверждает выполненную операцию.

## 9. Контрольные вопросы

1. Чем копирование отличается от перемещения файла?
2. Что происходит с исходным файлом после copy2()?
3. Что происходит с исходным путём после move()?
4. Для чего используется Path.rename()?
5. Зачем перед операцией проверять destination.exists()?
6. Почему все операции выполняются только внутри собственной рабочей папки?
7. Как проверить, что переименование действительно выполнено?

8. Почему после копирования могут одновременно существовать два файла с одинаковым содержимым?

## 10. Критерии готовности

| Проверка | Да / нет |
|---|---|
| Структура создана только внутри Path.cwd() | |
| Копирование выполнено и проверено по двум путям | |
| Переименование подтверждено через exists() | |
| Перемещение подтверждено через exists() | |
| Защитная проверка целевого пути работает | |
| Содержимое каталогов выведено через iterdir() | |
| Выполнено самостоятельное изменение | |
| В отчёте есть код, скриншоты, ответы и вывод | |